

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Model
CO Assessed:	CO4 – Precision (BL3, BL4, BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2*50=100
CO4 Statement:	Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)		
Student Name:	C. Praveen kumar	Reg. No.:	192472034
Section / Batch:	2024	Date:	20-08-2026

Code of Conduct

I C.Praveen kumar / 192472034 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C.Praveen kumar

Name: C.Praveen kumar

Reg.No: 192472034

Question 1: Tokenization

(a) Split "DISAGREEMENT" into 3 sub-word pieces

DIS + AGREE + MENT

Justification: This split separates the word into recognizable, reusable morphological units that a sub-word tokenizer (e.g., BPE or WordPiece) is likely to have learned from other words in its training data:

- "DIS" — a common negating prefix, also seen in "dislike," "disorder," "distrust."
- "AGREE" — the core root word, high-frequency on its own ("agree," "agreed," "agreeing").
- "MENT" — a common noun-forming suffix, also seen in "argument," "government," "enjoyment".

Because each piece independently appears in many other common words, the tokenizer's vocabulary is very likely to contain all three as learned merges, even if "DISAGREEMENT" itself never appeared as a whole token during training.

(b) Why sub-word tokenization helps

Sub-word tokenization lets the model represent any word — including rare or unseen ones — as a combination of smaller, frequently-seen pieces instead of needing every whole word in its vocabulary. Since the model has already learned the meaning and usage patterns of pieces like "DIS," "AGREE," and "MENT" from many other words, it can compose an understanding of "DISAGREEMENT" without ever having seen that exact word, which avoids the out-of-vocabulary problem that whole-word tokenizers face.

Question 2: Embedding

(a) Which pair has higher cosine similarity?

"dog" and "puppy" would have the **HIGHER** cosine similarity.

These two words are semantically related — they refer to the same animal (an adult dog vs. a young dog) and appear in very similar contexts (e.g., "walk the __," "feed the __," "pet the __"). "dog" and "truck" are unrelated in meaning and typically appear in very different contexts, so their embedding vectors would point in more different directions, giving a lower cosine similarity. Embedding models learn similarity largely from how words are used in surrounding context, so words that co-occur with similar neighboring words end up closer together in the vector space.

(b) What an embedding represents vs. a one-hot vector

An embedding is a dense vector of real numbers (typically hundreds of dimensions) that encodes a word's meaning based on the contexts it tends to appear in — words with similar meanings or usage patterns end up close together in this vector space, and the geometric relationships between vectors (distance, direction) can even capture semantic relationships like analogies. A one-hot vector, in contrast, is a sparse vector as long as the entire vocabulary, with a single 1 marking that word's index and 0s everywhere else; it treats every word as equally different from every other word and carries no information about meaning or similarity at all.

Question 3: Self attention Worked Example

"The city council refused the protesters a permit because they feared violence."

(a) "They" refers to:

✓ **City council**

(b) Why self-attention (not word order) is needed

A simple word-order rule (like "pronouns refer to the nearest preceding noun phrase") would incorrectly point "they" to "protesters," since that phrase is closer in the sentence. Resolving the reference correctly requires understanding the meaning of the whole sentence: it is the council's fear of violence that logically explains why they refused a permit, which is a semantic/contextual judgment rather than a positional one. Self-attention lets "they" directly attend to and weigh information from every other word in the sentence — regardless of distance — so the model can use cues like "refused" and "feared violence" to infer that "city council" is the more plausible antecedent.

[3.5 marks]

Question 4: Query, key, value

In self-attention, each input token is projected into three separate vectors — Query, Key, and Value — each capturing a different role in the attention computation:

- Query (Q): represents what the current token is "looking for" — the kind of information it needs from other tokens in order to update its own representation.
- Key (K): represents what each token "has to offer" — a label or signature that other tokens can match against when searching for relevant information.
- Value (V): represents the actual content or information that gets passed along once a token is deemed relevant — this is what actually gets aggregated into the output.

To combine them, the model computes a similarity score (typically a scaled dot product) between a token's Query vector and every other token's Key vector; these scores are passed through a softmax to produce attention weights that sum to 1. Each token's output representation is then a weighted sum of all the Value vectors, using those attention weights — so tokens whose Keys best match the Query contribute the most to the result.

[5 marks]

Practical Application: FAQ-Based College Chatbot Project

Name: C.Praveen kumar Reg.No: 192472034

The concepts covered in Questions 1–4 (tokenization, embeddings, self-attention, and Query/Key/Value) are not just theoretical — they are the exact mechanisms running inside the LLM backend of my own project, the **FAQ-Based College Chatbot** (Q10 of the assessment set). This section connects each worksheet answer to a concrete part of that project.

Project Overview

A Retrieval-Augmented Generation (RAG-lite) chatbot that answers 30+ FAQs on admissions, exams, library, placements, hostel, and IT support. It combines TF-IDF retrieval (fast, deterministic, explainable) with LLM generation (fluent, handles paraphrasing), and falls back gracefully when a question is outside the knowledge base. The LLM backend is provider-agnostic — it can run on Google Gemini, OpenAI, or a local Ollama model — all through a single **LLMClient.chat()** interface (*llm_client.py*).

Pipeline: TF-IDF Retrieval — scores the query against 30+ FAQ questions using cosine similarity, keeps the top-3 matches → Threshold Check — below 0.12 similarity triggers a fallback message instead of a guess → LLM Generation — Gemini / OpenAI / Ollama composes a grounded answer using only the retrieved FAQ context → Chat UI

Mapping Worksheet Concepts to the Project

Q1 – Tokenization

Every query the student types (e.g. "What's the hostel curfew?") and every FAQ answer pulled from *faq_data.json* is broken into sub-word tokens before it reaches Gemini/OpenAI. This is exactly why the bot handles college-specific or rare terms (e.g. "SIMATS", "CIA marks", hostel/placement jargon) even though the underlying tokenizer's vocabulary was never trained specifically on this institution's terminology — the same DIS + AGREE + MENT style sub-word composition lets it fall back on familiar pieces instead of failing on an out-of-vocabulary word.

Q2 – Embeddings vs. TF-IDF

The project deliberately uses **TF-IDF + cosine similarity** for retrieval rather than dense embeddings — a sparse, bag-of-words vector as discussed in Q2(b), not a learned dense one. This keeps retrieval fast, deterministic and explainable, but it is also the project's known limitation: TF-IDF can miss a paraphrase that uses completely different vocabulary for the same intent (e.g. "When is the college open?" vs. "What are the working hours?"). The README lists the direct fix as swapping in embedding-based retrieval (Gemini's *text-embedding-004* or a local *nomic-embed-text* model), which would place semantically similar questions like "dog" and "puppy" from Q2(a) close together in vector space even when they share no exact words.

Q3 – Self-Attention

Once the top-3 FAQ pairs are retrieved, they are concatenated into a single prompt alongside the user's question (*build_prompt()* in *app.py*) and handed to the LLM. Inside the LLM, self-attention is what lets it correctly ground the answer: it must connect pronouns and references in the user's question to the right entity inside the retrieved FAQ context, the same way "they" had to be resolved to "city council" using meaning rather than word order in Q3. This is also why the response stays grounded only in the supplied context instead of drifting to unrelated information.

Q4 – Query, Key, Value

When the LLM generates the final answer token by token, every token in the combined prompt (system instructions + retrieved FAQ context + user query) is projected into Query, Key and Value vectors. Each output token's Query attends over the Keys of the FAQ-context tokens, and the softmax-weighted Values pull the actual FAQ answer content into the response — this is the underlying mechanism that makes the "augment with retrieved context, then generate" step of the RAG-lite pipeline work at all.

Engineering Notes Relevant to the Concepts Above

- Multi-provider LLM support (Gemini / OpenAI / Ollama) behind one interface, so tokenization and attention behavior differ slightly by backend model but the retrieval layer stays identical.
- `SIM_THRESHOLD = 0.12` in *app.py* is the practical guardrail that prevents hallucinated answers when TF-IDF similarity (not semantic similarity) is too low.

- Empty input, out-of-scope queries, and API errors are all handled explicitly rather than left to the LLM to guess.

Files referenced: app.py, llm_client.py, config.py, faq_data.json, README.md (project archive: faq_chatbot.zip).

OUTPUT:

Settings

AI Provider

geminiv

FAQ answers are prioritized. General conversation is handled by the selected AI provider.

30+ FAQ topics covering admissions, exams, library, placements, hostel, and IT support.

Deploy

FAQ-Based College Chatbot

Backend: gemini - Model: gemini-3.6-flash

hi

explain about dl

Deep Learning (DL) is a specialized branch of Machine Learning (ML) and Artificial Intelligence (AI) that teaches computers to process data in a way inspired by the human brain.

Here is a quick breakdown of how it works and why it matters:

1. How It Works: Neural Networks

At the core of deep learning are Artificial Neural Networks.

- These networks consist of layers of interconnected nodes (or "neurons").
- Information passes through an input layer, through multiple hidden layers (which is why it's called deep learning), and finally to an output layer.
- As the network processes massive amounts of data, it adjusts itself to recognize complex patterns and make predictions.

Ask about admissions, exams, library, placements or anything else...



FAQ-Based College Chatbot

Backend: gemini · Model: gemini-3.6-flash



hi



explain about dl



Deep Learning (DL) is a specialized branch of **Machine Learning (ML)** and **Artificial Intelligence (AI)** that teaches computers to process data in a way inspired by the human brain.

Here is a quick breakdown of how it works and why it matters:

1. How It Works: Neural Networks

At the core of deep learning are **Artificial Neural Networks**.

- These networks consist of layers of interconnected nodes (or "neurons").
- Information passes through an **input layer**, through multiple **hidden layers** (which is why it's called *deep learning*), and finally to an **output layer**.
- As the network processes massive amounts of data, it adjusts itself to recognize complex patterns and make predictions.

Ask about admissions, exams, library, placements or anything else...

